

报告篇幅	报告内容	排版规范	参考文献	课程总结	总分
报告得分（教师填写）					

“软件工程导论” 报告

(2026)

姓 名： 吕舒君

班 级： 软国 2508

学 号： 20252111170



大连理工大学

DALIAN UNIVERSITY OF TECHNOLOGY

AI 时代的后端技术变革：Vibe Coding 与 AI 辅助开发实践

吕舒君，20252111170

摘要：

随着大语言模型的快速发展，Vibe Coding 作为一种以自然语言提示词为核心的全新开发方式，正在对传统后端编程造成了巨大冲击。本文系统地探讨了 Vibe Coding 在 AI 时代后端开发中的理论逻辑、实践应用，同时结合作者的 Go 语言项目开发实例，验证了 Vibe Coding 在数据库设计与中间件开发中的显著效率优势。同时，本文深入剖析了 Vibe Coding 在安全隐私与性能可维护性方面的隐患，揭示了大语言模型具有的潜在风险，为开发者盲目依赖 Vibe Coding 提出了警告。本文综合论述 Vibe Coding 的各方面特点以及实践表现，为开发者能够有效运用 Vibe Coding 提出启发。

关键词：大模型；Vibe Coding；AI 辅助编码；后端开发；可维护性

1 概述

随着大语言模型的不断演进，在完成代码任务方面，它由最初的根据提示词直接生成代码，演变为通过 Agent 形式介入整体项目持续编码。开发者由手动编写代码，逐渐转向通过撰写包含软件基本功能的文档和提示词，而后使用大语言模型生成并完善整个软件工程项目^[1]。此时，氛围编程（Vibe Coding）应运而生。这是一种以自然语言提示词为核心的软件开发方法。基本的开发流程即为开发者描述需求，AI 助手生成并完善代码，并以此重复，直到程序功能满足开发者的预期开发目标。这一概念由 AI 研究者 Andrej Karpathy 于 2025 年提出。在当时的那条推特帖子中，他指出：“这就是一种完全沉浸于氛围、接受 AI 的多种可能性、甚至忘了代码的存在的编程方式。我认为可能的，因为大型语言模型（比如 Cursor Composer 和 Sonnet）发展得太快。这使得我只需要和他们对话，几乎不怎么需要碰键盘来修改代码。”^[2] 根据 Google Cloud 的文档描述，Vibe Coding 将“开发者的主要角色从逐行编写代码转变为通过对话指导 AI 生成、完善和调试应用”。它让应用构建变得更加容易，使缺乏传统编程经验的人员也能够轻松地创建属于他们自己的应用，从而极大地降低了编程门槛并加速原型开发。在后端领域，Vibe Coding 尤其适合快速原型开发和自动化重复性任务，例如生成 API 接口、数据库操作等逻辑。通过 Vibe Coding，后端开发时可以将更多精力放在架构设计与业务流程上，让 AI 承担具体的代码实现工作。

2 核心逻辑

2.1 “意图驱动”的开发模式

Vibe Coding 的本质是一种“意图驱动”的开发模式。它不再追求一行行代码的单独编写，而是转而追求开发意图的优先实现。在传统编程的流程中，开发者通常遵循“需求分析-设计-编码-调试-测试”的线性流程。开发者既是设计者也是执行者，需要通过手工编写代码来实现预定的开发目标。而对于 Vibe Coding 介入的开发流程，开发流程被压缩为“描述意图-AI 生成代码-验证结果-迭代优化”的循环流程，开发者不再需要手动设计代码，而仅仅是对 AI 生成的开发产物进行测试验证，并进一步发出指令来完善开发产物。因此，微软在官方培训文档中将 Vibe Coding 定义为“一种 AI 驱动的软件开发形式，重点从逐行编写代码转向以自然语言描述所需产品体验”^[3]。

2.2 生成的不确定性与可能性

在传统编程实践中，代码的输出往往都是唯一确定的。对于同一段源代码，在绝大多数环境下的编译运行结果都应当保持一致，这也构成了跨平台开发与交叉编译的基础之一。然而 Vibe Coding 的编码过程中由于引入了大语言模型。我们熟知，主流的大语言模型是通过下一词元预测（Next-Token Prediction）算法来返回结果的^[4]。因此 Vibe Coding 的使用便引入了一种非确定性生成机制，即同一段自然语言描述在不同时刻、不同模型版本下可能产生不同代码。这使得开发流程更接近于一种对概率的抽奖“老虎机”，即开发者不断调整提示词以逼近理想结果，而非一次性完成编码^[5]。因此，Vibe Coding 的编码质量除了取决于大模型本身的能力边界外，也高度依赖于开发者与大模型之间的交互质量。同时，Vibe Coding 后的测试与验证也显得尤为关键。

2.3 角色转型：从工匠到指挥家

Vibe Coding 引入编程后带来的核心变革在于开发者角色的转换。在传统编程模式下，开发者更加类似于掌握工具的用法的工匠，亲手打磨每一行代码。而在 Vibe Coding 模式下，开发者更像是指挥家。开发者仅仅需要完成定义目标、拆解任务、评估结果、发出修正指令等指令性操作，而具体的代码实现交由 AI 完成。这一转变对后端开发意义深远：开发者不再被困于重复的 CRUD 实现、配置文件的编写或样板代码的填充，而是能够将精力集中于诸如后端架构优化、性能增强与质量把关等更高层次的工程活动^[6]。研究表明，通过 Vibe Coding 生成高质量代码的关

键并非在于开发者自身的编码能力，而是更加考验开发者对上下文的理解能力和需求拆解能力。因此，这对开发者的系统化思维提出了更高要求。

3 市场应用

随着 Vibe Coding 问世以来，市场上也出现了诸多 Vibe Coding 工具，部分工具已形成差异化竞争格局。以下选取 Claude Code^[7]、GitHub Copilot^[8]、Trae^[9] 三个具有代表性的工具进行横向对比：

产品名称	Claude Code	GitHub Copilot	Trae
厂商	Anthropic	Microsoft/GitHub	字节跳动
终端	控制台 CLI	VSCode 插件	Trae（VS Code 分支）
大模型	Claude Opus 4.6	GPT-5.4 / o3-mini	豆包 1.5 Pro / DeepSeek R1
SWE-bench 得分	80.8%	65.2%	58.3%
上下文窗口	200 万 Token	50 万 Token	100 万 Token
Agent 自主性	自主规划、工具调用、自动重试修复	简单任务、需人工确认	基础规划、需较多人工介入
核心优势	推理能力强，Agent 自主性最高，工具调用能力强	与 VSCode 深度集成，补全速度最快，上手门槛低	国内完全免费，中文支持优秀，SOLO 模式同时支持完成非编程任务
主要缺点	命令行交互不直观，不适合实时补全，成本较高	上下文窗口小，跨文件理解弱，Agent 自主性一般	复杂任务出错率较高，模型能力有限
价格	Pro \$20/月	Pro \$10/月	国内版免费

产品名称	Claude Code	GitHub Copilot	Trae
适用场景	复杂重构、大项目设计、大规模代码迁移	日常编码补全、企业标准化部署	国内个人开发者、初级学习、小项目开发

4 开发实践

4.1 数据库结构设计

在传统后端开发中，数据库结构设计是项目启动阶段的关键部分之一。开发者需要根据预先分析的业务需求，手动定义数据表结构、字段类型、索引、主键和关联关系，这一过程需要较长的设计时间，且要求设计者具备扎实的数据库理论功底和较高的抽象能力。在数据库结构设计阶段引入 Vibe Coding 将改变这一局面。引入后，开发者只需用自然语言描述业务内容及其各个细分功能，AI 便可生成相应的数据定义语言（DDL）和对象关系映射（ORM）模型。

此处以我最近使用 Go 开发的 latexq-app (<https://github.com/Lvshujun0918/latex-qapp>) 为例。我使用 Copilot 进行 Vibe Coding 开发，向它描述“我需要一个记录 PDF 生成记录的数据表”。联系我的项目结构与使用的框架，AI 即可快速给出准确的 Gorm 代码：

```
1 package model
2
3 import "time"
4
5 type PDFJob struct {
6     ID uint `gorm:"primaryKey" json:"id"`
7     UserID uint `gorm:"index;not null" json:"user_id"`
8     JobID string `gorm:"size:64;uniqueIndex;not null" json:"job_id"`
9     Status string `gorm:"size:16;not null;default:queued" json:"status"`
10    Progress int `gorm:"not null;default:0" json:"progress"`
11    SelectedCount int `gorm:"not null;default:0" json:"selected_count"`
12    PDFFileURL string `gorm:"size:512" json:"pdf_file_url"`
13    Message string `gorm:"size:255" json:"message"`
14    Questions []PDFJobRecord `gorm:"foreignKey:JobID;references:JobID"
15        json:"questions,omitempty"`
16    CreatedAt time.Time `json:"created_at"`
17    UpdatedAt time.Time `json:"updated_at"`
18 }
19
20 type PDFJobRecord struct {
```

```

20     ID uint `gorm:"primaryKey" json:"id"`
21     JobID string `gorm:"size:64;index;not null" json:"job_id"`
22     UserID uint `gorm:"index;not null" json:"user_id"`
23     RecordID uint `gorm:"index;not null" json:"record_id"`
24     Index int `gorm:"not null" json:"index"`
25     Title string `gorm:"size:255" json:"title"`
26     Subject string `gorm:"size:32" json:"subject"`
27     QuestionType string `gorm:"size:32" json:"question_type"`
28     LatexSource string `gorm:"type:text" json:"latex_source"`
29     LatexAnswer string `gorm:"type:text" json:"latex_answer"`
30     ChildResult string `gorm:"size:16;not null;default:none" json:"
        child_result"`
31     ReviewedAt time.Time `json:"reviewed_at"`
32     CreatedAt time.Time `json:"created_at"`
33     UpdatedAt time.Time `json:"updated_at"`
34 }

```

可见以上 AI 生成的代码流畅清晰，符合专业的 DBA 设计师的设计，已经达到了很高的编程水平。

4.2 中间件设计

中间件是后端架构中实现流量验证、拦截、转发的核心机制，它通常具备实现身份认证、限流熔断、日志追踪、跨域处理等功能，是项目权限控制和记录的关键所在^[10]。传统开发中，中间件的编写需要遵循固定的“轮子”，涉及大量模板化代码的重复编写。由于中间件天然具有功能接口统一，格式固定的特性，与 AI 擅长生成规范化、标准化代码的优势相契合，因此使用 Vibe Coding 生成十分便捷。

此处仍然以我开发的 latexq-app (<https://github.com/Lvshujun0918/latex-qapp>) 为例，我要求 AI “生成一个用于后端权限认证与拦截的中间件”，它生成了如下代码：

```

1  package middleware
2
3  import (
4      "net/http"
5      "strings"
6
7      "github.com/gin-gonic/gin"
8      "github.com/golang-jwt/jwt/v5"
9  )
10
11 func JWTAuth(secret string) gin.HandlerFunc {
12     key := []byte(secret)
13     return func(c *gin.Context) {

```

```
14     auth := c.GetHeader("Authorization")
15     if !strings.HasPrefix(auth, "Bearer ") {
16         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"ok": false
17             , "error": "missing bearer token"})
18         return
19     }
20     tokenString := strings.TrimPrefix(auth, "Bearer ")
21     token, err := jwt.Parse(tokenString, func(token *jwt.Token) (any,
22         error) {
23         return key, nil
24     })
25     if err != nil || !token.Valid {
26         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"ok": false
27             , "error": "invalid token"})
28         return
29     }
30     claims, ok := token.Claims.(jwt.MapClaims)
31     if !ok {
32         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"ok": false
33             , "error": "invalid claims"})
34         return
35     }
36     userIDFloat, ok := claims["sub"].(float64)
37     if !ok {
38         c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{"ok": false
39             , "error": "invalid sub"})
40         return
41     }
42     c.Set("userID", uint(userIDFloat))
43     c.Next()
44 }
```

可见此处 AI 实现了完整的 JWT 校验和用户 ID 参数提取，流程顺畅简洁，达到了较高的编程水平。

5 隐患与缺陷

5.1 安全与隐私

后端涉及敏感代码的编程场景中，使用 Vibe Coding 很有可能带来额外的安全风险。若不经代码审查，轻易采用 AI 生成的代码，则关键业务代码容易引入例如 SQL 注入或权限控制失效等漏洞，这会极大地削弱原有系统的安全性。此外，盲目使用 Vibe Coding 还存在“提示词诱饵”的风险，例如 AI 提示词注入攻击。攻击者通过在项目中预先埋放含有恶意提示词的 Markdown 文件，Vibe Coding 流程中 AI 有可能读取并使用此文件中的内容，造成生成的代码中存在预定义的后门、漏洞，利于被黑客利用^[11]。此外，在隐私方面，Vibe Coding 通常需要通过 API 将需求和部分数据发送给第三方模型（如 OpenAI），这会导致数据通过网络传输，存在较高的敏感数据泄露风险。

5.2 性能与可维护性

由于涉及远程模型调用，Vibe Coding 流程存在比传统开发显著提升的性能开销与环境依赖，首先它需要特定的网络环境，在离线状态下完全无法开展工作；此外，Vibe Coding 在调用大模型时通常通过词元数计费，对于较高质量的模型，其每 Token 的定价较高，若盲目多次使用 Vibe Coding 会显著增加项目开发成本^[12]。

Vibe Coding 生成的代码有时缺乏结构化注释，这使得后续进一步进行人工维护和二次开发变得困难。而通过人工补足注释又将会带来额外的时间成本，导致 Vibe Coding 原先的敏捷开发特点丢失。

5.3 结论

在全民 AI 的时代背景下，Vibe Coding 作为大语言模型驱动的新型开发范式，正在从底层重塑后端开发流程。本文研究表明，其核心价值在于将开发者从重复性编码劳动中解放，聚焦于更高层次的架构、算法设计，给开发者的个人能力提升提供可能。通过本人的项目在数据库设计与中间件开发的实践验证可见，Vibe Coding 在模板化、规范化的后端任务中具有显著的效率优势。然而，AI 的不确定性带来的安全漏洞、隐私泄露与可维护性隐患同样应当被我们重视。展望未来，Vibe Coding 的理想实践路径不应当是 100% 依赖 AI 进行盲目开发，而是应当建立起“约束定义

“一生成验证—人工加固”的人机分层协作体系。只有在开发效率与代码之间找到一个恰当的平衡点，Vibe Coding 才能真正释放其变革潜力，推动后端开发迈向更高层次的智能化协作。

6 参考文献

- [1] FAWZY A, TAHIR A, BLINCOE K. Vibe Coding in Practice: Motivations, Challenges, and a Future Outlook – a Grey Literature Review[J/OL]. arXiv preprint, 2025, arXiv:2510.00328. <https://arxiv.org/abs/2510.00328>.
- [2] KARPATY A. Vibe Coding Post[M]. <https://x.com/karpathy/status/1886192184808149383>, 2025.
- [3] TRAINING M. 什么是 Vibe 编码? - 微软开发者培训 [EB/OL]. (2023). <https://learn.microsoft.com/zh-cn/training/modules/introduction-vibe-coding/2-what-vibe-coding>.
- [4] LI Y, HUANG Y, ILDIZ M E, 等. Mechanics of Next Token Prediction with Self-Attention[C/OL]//DASGUPTA S, MANDT S, LI Y. Proceedings of The 27th International Conference on Artificial Intelligence and Statistics: 卷 238. PMLR, 2024: 685-693. <https://proceedings.mlr.press/v238/li24f.html>.
- [5] RAY P P. A Review on Vibe Coding: Fundamentals, State-of-the-art, Challenges and Future Directions[J/OL]. TechRxiv, 2025, 2025(0509). <https://www.techrxiv.org/doi/abs/10.36227/techrxiv.174681482.27435614/v1>.
- [6] SETIAWAN ismail, ARTHA F D, IKTISOM R W A. PENINGKATAN KEMAMPUAN CODING ANAK USIA REMAJA DENGAN METODE CRUD GENERATOR BERBASIS WEB DENGAN ANALISA DATABASE[J/OL]. PEDAMAS (PENGABDIAN KEPADA MASYARAKAT), 2023, 1(2): 331-337. <https://pekatpkm.my.id/index.php/JP/article/view/59>.
- [7] @CLAUDEAI. Claude Platform | Claude[EB/OL]. <https://claude.com/platform/api>.
- [8] GITHUB M. GitHub Copilot · Your AI pair programmer[EB/OL]. <https://github.com/features/copilot>.
- [9] 字节跳动. Trae | 智能无限，协作无间 [EB/OL]. <https://www.trae.cn/>.
- [10] 吴泉源. 网络计算中间件[J]. 软件学报, 2013, 24(1): 67.

- [11]ZHAO S, WANG D, ZHANG K, 等. Is Vibe Coding Safe? Benchmarking Vulnerability of Agent-Generated Code in Real-World Tasks[EB/OL]. (2026). <https://arxiv.org/abs/2512.03262>.
- [12]PATTYN F, GOETZ P. The Hidden Cost of Vibe Coding: Rework, Rotations and Real Startup Time-to-Market[J].